

A Predictive Maintenance Framework for Autonomous Flight Systems using High-Dimensional Sensor Log Pattern

Name: Bhatia Parth Nileshkumar

Enrolment: 25MML0018

Dataset Overview

1. What is it about?

The dataset contains continuous, time-series flight telemetry data simulating aircraft sensor readings. It captures detailed operational metrics across different aircraft (identified by callsigns like VIT-AI-400, VIT-AI-500) as they progress through standard flight phases (TAKEOFF, CLIMB, CRUISE, and DESCENT). The data serves as a foundation for monitoring aircraft health and performance.

2. What attributes does it contain?

The JSON data is nested and flattens into the following key attributes:

- **Temporal & Identifier Data:**
 - `timestamp`: The exact UTC time of the sensor reading.
- **Flight Info (`flight_info`):**
 - `callsign`: The aircraft's identifier.
 - `airframe_hours`: Total accumulated flight hours for the airframe.
 - `flight_phase`: The current operational phase (e.g., TAKEOFF, CLIMB, CRUISE, DESCENT).
- **Kinematics (`kinematics`):**
 - `altitude_ft`: The aircraft's altitude in feet.
 - `airspeed_knots`: The current airspeed in knots.
 - `g_force_axial`: The axial G-force acting on the aircraft.
- **Propulsion System (`propulsion_sys`):**
 - `engine_1_egt_c`: Exhaust Gas Temperature in Celsius.
 - `engine_1_vibration`: Engine vibration levels.
 - `fuel_remaining_lbs`: The amount of fuel left in pounds.
- **Analytics Metadata (`analytics_meta`):**
 - `maintenance_flag`: A binary indicator (0 or 1) highlighting whether the aircraft exhibits states requiring maintenance.

3. What problem statement are you solving?

The primary problem statement is **predictive maintenance and anomaly detection via unsupervised learning**. By applying clustering algorithms to the kinematics and

propulsion metrics, you are grouping flight telemetry into distinct operational states. The goal is to isolate "High Risk" clusters (e.g., groupings with elevated engine_1_vibration or abnormal Exhaust Gas Temperatures) so that maintenance teams can proactively identify aircraft approaching critical thresholds before a mechanical failure occurs.

Code:

```
import pandas as pd
import json
```

```
with open("balanced_flight_telemetry.json") as f:
    data = json.load(f)
```

```
df = pd.json_normalize(data)
print(df.head())
```

```

      timestamp flight_info.callsign  flight_info.airframe_hours
\
0  2026-04-09T10:00:00Z          VIT-AI-400          14250.500000
1  2026-04-09T10:00:01Z          VIT-AI-400          14250.500278
2  2026-04-09T10:00:02Z          VIT-AI-400          14250.500556
3  2026-04-09T10:00:03Z          VIT-AI-400          14250.500833
4  2026-04-09T10:00:04Z          VIT-AI-400          14250.501111

      flight_info.flight_phase  kinematics.altitude_ft  kinematics.airspeed_kn
ots \
0          TAKEOFF                0.0                280
.52
1          TAKEOFF                0.0                282
.23
2          TAKEOFF                0.0                278
.10
3          TAKEOFF                0.0                279
.41
4          TAKEOFF                0.0                281
.05

      kinematics.g_force_axial  propulsion_sys.engine_1_egt_c  \
0                0.990                638.90
1                1.000                635.62
2                1.002                633.70
3                0.999                633.46
4                0.999                633.96

      propulsion_sys.engine_1_vibration  propulsion_sys.fuel_remaining_lbs  \
0                0.3067                18000.0
1                0.3077                17998.5
2                0.2999                17997.0
3                0.3280                17995.5
4                0.3241                17994.0
```

```

    analytics_meta.maintenance_flag
0      0
1      0
2      0
3      0
4      0

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import json

# 1. Load the data
with open('balanced_flight_telemetry.json', 'r') as f:
    data = json.load(f)

# 2. Flatten JSON into a DataFrame
df = pd.json_normalize(data)

# Create a figure with two subplots side-by-side
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# --- GRAPH 1: Class Distribution (Bar Chart) ---
sns.countplot(x='analytics_meta.maintenance_flag', data=df, ax=axes[0], palette='viridis')
axes[0].set_title('Class Distribution (Maintenance Flag)', fontsize=14)
axes[0].set_xlabel('0 = Normal | 1 = Anomaly')
axes[0].set_ylabel('Record Count')

# Add text labels on the bars
for p in axes[0].patches:
    axes[0].annotate(f'{int(p.get_height())}', (p.get_x() + 0.35, p.get_height() + 50))

# --- GRAPH 2: Normal Distribution (Sensor Bell Curves) ---
# We plot the vibration distribution for the "Normal" state (flag 0)
# to see the clean Gaussian curve before the anomaly drift begins.
normal_data = df[df['analytics_meta.maintenance_flag'] == 0]

sns.histplot(normal_data['propulsion_sys.engine_1_vibration'], kde=True, ax=axes[1], color='blue')
axes[1].set_title('Normal Distribution of Engine Vibration (Class 0)', fontsize=14)
axes[1].set_xlabel('Vibration Level')
axes[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()

# Print Summary
print("--- Data Summary ---")
print(f"Total Vehicles: {df['flight_info.callsign'].nunique()}")

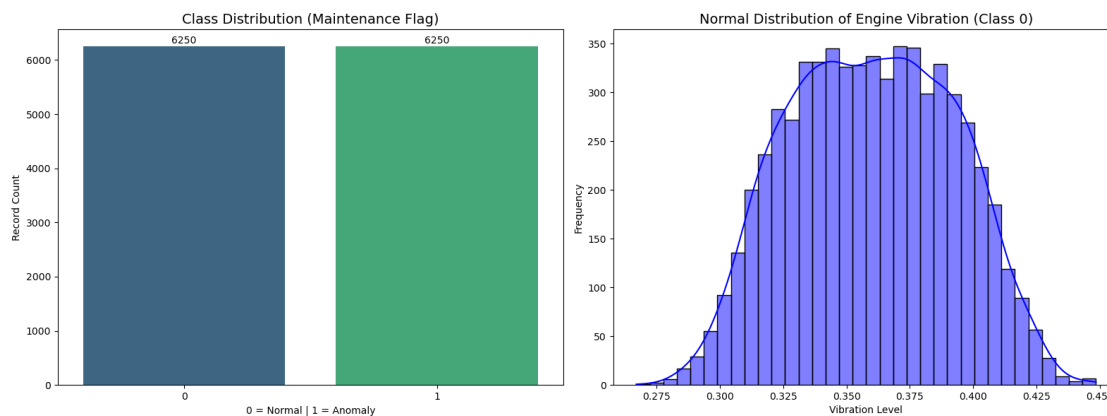
```

```
print(f"Class Counts:\n{df['analytics_meta.maintenance_flag'].value_counts()})")
```

C:\Users\Parth\AppData\Local\Temp\ipykernel_19704\3538527215.py:17: Future Warning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='analytics_meta.maintenance_flag', data=df, ax=axes[0], palette='viridis')
```



--- Data Summary ---

Total Vehicles: 5

Class Counts:

```
analytics_meta.maintenance_flag
```

```
0    6250
```

```
1    6250
```

```
Name: count, dtype: int64
```

Preprocessing

```
# =====
# PREPROCESSING STEPS
# =====
```

```
# 1. Check basic info
```

```
print("Dataset Info:")
```

```
print(df.info())
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

```
# 2. Handle Missing Values
```

```
# Fill numeric columns with mean
```

```
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

```
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
```

```
# Fill categorical columns with mode
```

```
categorical_cols = df.select_dtypes(include=['object']).columns
```

```

for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# 3. Remove Duplicates
df.drop_duplicates(inplace=True)

# 4. Convert Timestamp to Datetime
if 'timestamp' in df.columns:
    df['timestamp'] = pd.to_datetime(df['timestamp'])

# 5. Feature Engineering (optional but useful)
# Example: Extract hour from timestamp
if 'timestamp' in df.columns:
    df['hour'] = df['timestamp'].dt.hour

# 6. Encode Categorical Variables
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
for col in categorical_cols:
    try:
        df[col] = le.fit_transform(df[col])
    except:
        pass # skip complex nested fields

# # 7. Normalize / Scale Data
# from sklearn.preprocessing import StandardScaler

# scaler = StandardScaler()
# df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

# 8. Handle Outliers (Optional)
# Remove extreme values using IQR
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    df = df[(df[col] >= Q1 - 1.5 * IQR) & (df[col] <= Q3 + 1.5 * IQR)]

print("\nPreprocessing Completed ✅")
print(df.head())

```

Dataset Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 12500 entries, 0 to 12499

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	timestamp	12500 non-null	object
1	flight_info.callsign	12500 non-null	object
2	flight_info.airframe_hours	12500 non-null	float64
3	flight_info.flight_phase	12500 non-null	object
4	kinematics.altitude_ft	12500 non-null	float64

```

5   kinematics.airspeed_knots      12500 non-null float64
6   kinematics.g_force_axial       12500 non-null float64
7   propulsion_sys.engine_1_egt_c   12500 non-null float64
8   propulsion_sys.engine_1_vibration 12500 non-null float64
9   propulsion_sys.fuel_remaining_lbs 12500 non-null float64
10  analytics_meta.maintenance_flag 12500 non-null int64
dtypes: float64(7), int64(1), object(3)
memory usage: 1.0+ MB
None

```

Missing Values:

```

timestamp                0
flight_info.callsign      0
flight_info.airframe_hours 0
flight_info.flight_phase  0
kinematics.altitude_ft    0
kinematics.airspeed_knots 0
kinematics.g_force_axial  0
propulsion_sys.engine_1_egt_c 0
propulsion_sys.engine_1_vibration 0
propulsion_sys.fuel_remaining_lbs 0
analytics_meta.maintenance_flag 0
dtype: int64

```

Preprocessing Completed ☒

```

timestamp  flight_info.callsign  flight_info.airframe_hours \
0          0                    0          14250.500000
1          1                    0          14250.500278
2          2                    0          14250.500556
3          3                    0          14250.500833
4          4                    0          14250.501111

```

```

flight_info.flight_phase  kinematics.altitude_ft \
0                          3                      0.0
1                          3                      0.0
2                          3                      0.0
3                          3                      0.0
4                          3                      0.0

```

```

kinematics.airspeed_knots  kinematics.g_force_axial \
0                          280.52                    0.990
1                          282.23                    1.000
2                          278.10                    1.002
3                          279.41                    0.999
4                          281.05                    0.999

```

```

propulsion_sys.engine_1_egt_c  propulsion_sys.engine_1_vibration \
0                              638.90                    0.3067
1                              635.62                    0.3077
2                              633.70                    0.2999
3                              633.46                    0.3280
4                              633.96                    0.3241

```

```

propulsion_sys.fuel_remaining_lbs  analytics_meta.maintenance_flag  hou

```

```

r
0          18000.0          0    1
0
1          17998.5          0    1
0
2          17997.0          0    1
0
3          17995.5          0    1
0
4          17994.0          0    1
0

```

C:\Users\Parth\AppData\Local\Temp\ipykernel_19704\2707193242.py:20: Future Warning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```

df[col].fillna(df[col].mode()[0], inplace=True)
print(df.columns)
Index(['timestamp', 'flight_info.callsign', 'flight_info.airframe_hours',
      'flight_info.flight_phase', 'kinematics.altitude_ft',
      'kinematics.airspeed_knots', 'kinematics.g_force_axial',
      'propulsion_sys.engine_1_egt_c', 'propulsion_sys.engine_1_vibration',
      'propulsion_sys.fuel_remaining_lbs', 'analytics_meta.maintenance_flag',
      'hour'],
      dtype='object')

df = df.drop('flight_info.callsign', axis=1)

```

2. Convert timestamp → useful features

```

df['timestamp'] = pd.to_datetime(df['timestamp'])

df['hour'] = df['timestamp'].dt.hour
df['day'] = df['timestamp'].dt.day
df['month'] = df['timestamp'].dt.month
df['day_of_week'] = df['timestamp'].dt.dayofweek

```

3. Encode flight phase

```
from sklearn.preprocessing import LabelEncoder
```

```

le = LabelEncoder()
df['flight_phase_encoded'] = le.fit_transform(df['flight_info.flight_phase'])

```

```
df = df.drop('flight_info.flight_phase', axis=1)
```

4. Create ENGINE HEALTH features

```
df['engine_stress'] = (
    df['propulsion_sys.engine_1_egt_c'] *
    df['propulsion_sys.engine_1_vibration']
)
```

5. Create FUEL efficiency feature

```
df['fuel_efficiency'] = (
    df['kinematics.airspeed_knots'] /
    (df['propulsion_sys.fuel_remaining_lbs'] + 1)
)
```

6. Create INTERACTION features

```
df['altitude_stress'] = df['kinematics.altitude_ft'] * df['engine_stress']
df['speed_stress'] = df['kinematics.airspeed_knots'] * df['engine_stress']
```

7. Drop timestamp

```
df = df.drop('timestamp', axis=1)
```

Dimensionality Reduction Using PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
features = df.select_dtypes(include=[np.number]).columns.tolist()
```

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[features])
```

Reduce features down to 3 Principal Components

```
pca = PCA(n_components=3)
pca_data = pca.fit_transform(scaled_data)
```

Calculate how much information kept

```
variance_ratio = np.sum(pca.explained_variance_ratio_)
print(f"Variance captured by 3 components: {variance_ratio*100:.2f}%")
```

```
pca_df = pd.DataFrame(pca_data, columns=['PC1', 'PC2', 'PC3'])
```

ADDED 3D VISUALIZATION

```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
```

Create the 3D scatter plot

```
scatter = ax.scatter(
    pca_df['PC1'],
    pca_df['PC2'],
    pca_df['PC3'],
    c='teal',          # The color mapping
```



```

    cmap='viridis',    # Color template for clusters if 'c' is an array
    alpha=0.6,         # Transparency
    s=30               # Dot size
)

# Labeling titles and adding dynamic axes percentages
ax.set_title('3D PCA Dimensionality Reduction', fontsize=15, pad=15)
ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)', labelpad=10)
ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)', labelpad=10)
ax.set_zlabel(f'PC3 ({pca.explained_variance_ratio_[2]*100:.1f}%)', labelpad=10)

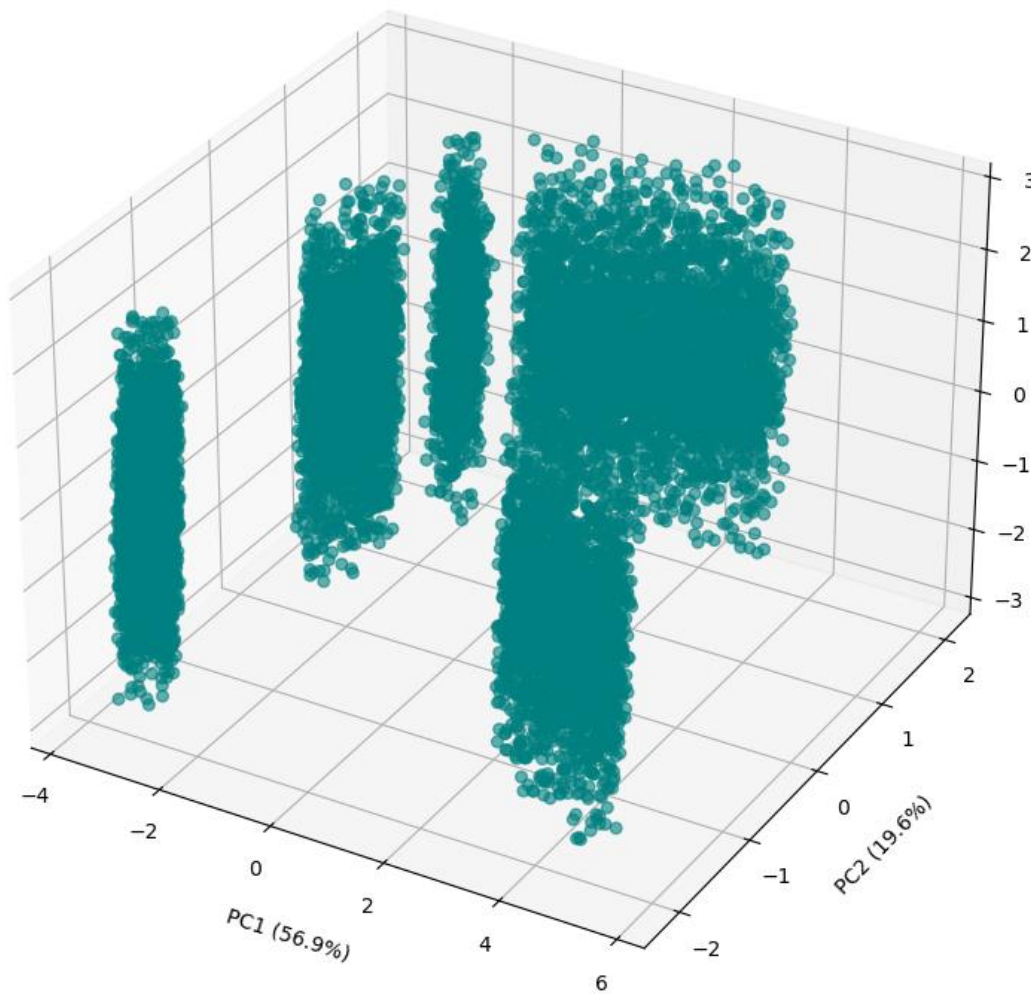
plt.tight_layout()
plt.show()

Variance captured by 3 components: 84.26%

C:\Users\Parth\AppData\Local\Temp\ipykernel_19704\158958470.py:27: UserWarning: No data for colormapping provided via 'c'. Parameters 'cmap' will be ignored
    scatter = ax.scatter(

```

3D PCA Dimensionality Reduction



The Clustering Model

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster_label'] = kmeans.fit_predict(scaled_data)

pca_df['cluster'] = df['cluster_label']

if 'flight_info.callsign' in df.columns:
    pca_df['flight_id'] = df['flight_info.callsign']
else:
    pca_df['flight_id'] = df.index
```

```

pattern_map = {0: 'Healthy', 1: 'Transitioning', 2: 'Critical'}
pca_df['pattern'] = pca_df['cluster'].map(pattern_map)

print("Clustering complete. Flight patterns identified.")

fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

colors = {'Healthy': '#2ecc71', 'Transitioning': '#f1c40f', 'Critical': '#e74c3c'}

for pattern, color in colors.items():
    subset = pca_df[pca_df['pattern'] == pattern]
    ax.scatter(
        subset['PC1'],
        subset['PC2'],
        subset['PC3'],
        c=color,
        label=pattern,
        alpha=0.6,
        s=40,
        edgecolors='w'
    )

ax.set_title('3D Flight Behavior Clusters', fontsize=16, fontweight='bold')
ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
ax.set_zlabel(f'PC3 ({pca.explained_variance_ratio_[2]*100:.1f}%)')

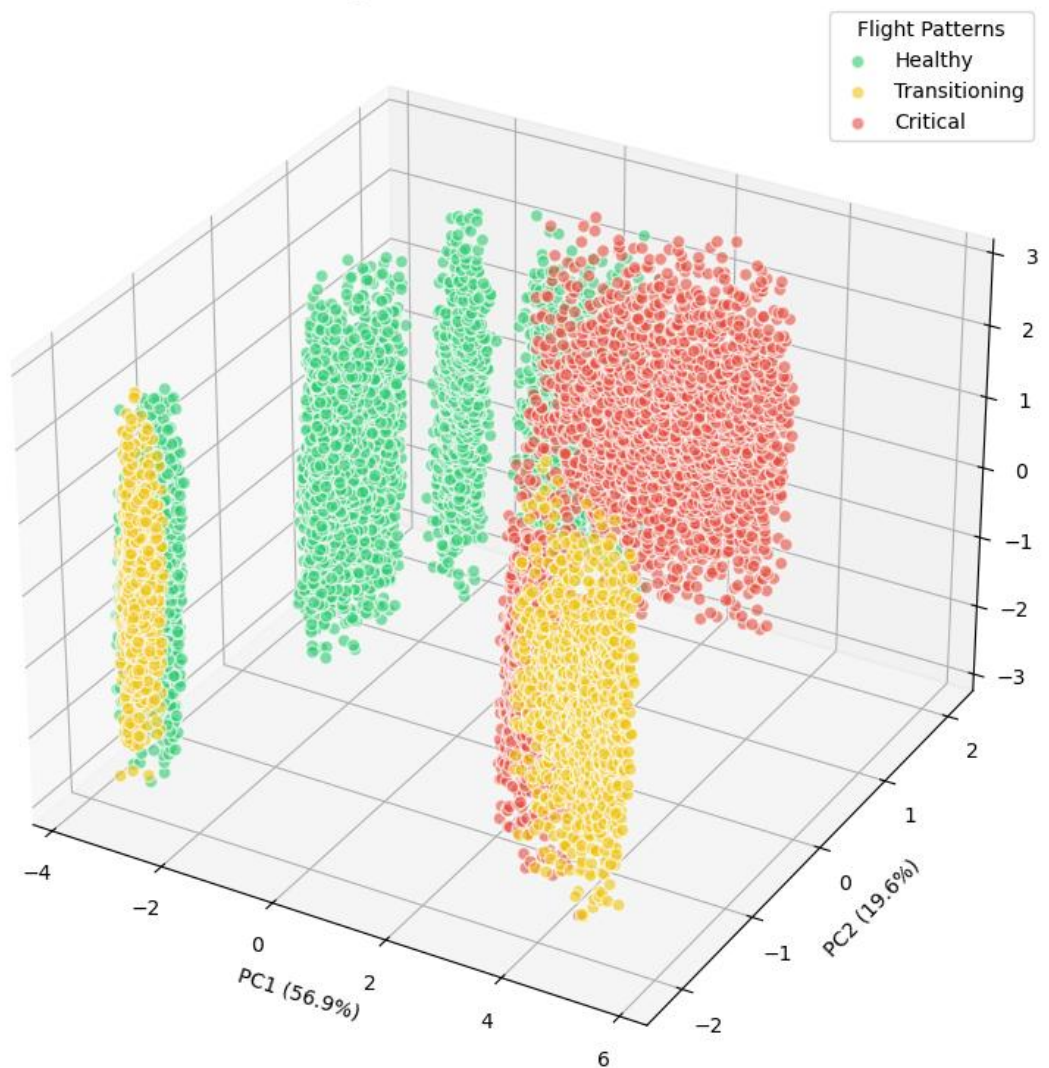
ax.legend(title="Flight Patterns", loc='best')
plt.tight_layout()
plt.show()

print("\nFlight Pattern Count:")
print(pca_df['pattern'].value_counts())

Clustering complete. Flight patterns identified.

```

3D Flight Behavior Clusters



Flight Pattern Count:

```
pattern
Healthy      6085
Critical      3616
Transitioning  2060
Name: count, dtype: int64
```

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
```

```
df = df.reset_index(drop=True)
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(scaled_data)
```

```

pca_df['cluster'] = cluster_labels

if 'flight_info.callsign' in df.columns:
    pca_df['flight_id'] = df['flight_info.callsign'].values
else:

    pca_df['flight_id'] = [f"Flight-{i}" for i in range(len(pca_df))]

print("Clustering complete. Index alignment verified.")

Clustering complete. Index alignment verified.

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# 1. Setup the Visualization
fig = plt.figure(figsize=(12, 9))
ax = fig.add_subplot(111, projection='3d')

# 2. Define High-Contrast Colors for Impact
# Cluster 0 = Healthy (Green), Cluster 1 = Warning (Yellow), Cluster 2 = Critical (Red)
colors = {0: '#27ae60', 1: '#f1c40f', 2: '#e74c3c'}
labels = {0: 'Healthy Operation', 1: 'Maintenance Warning', 2: 'Critical Failure Risk'}

# 3. Plot each cluster to ensure they appear in the Legend
for cluster_id in sorted(pca_df['cluster'].unique()):
    subset = pca_df[pca_df['cluster'] == cluster_id]
    ax.scatter(
        subset['PC1'], subset['PC2'], subset['PC3'],
        c=colors[cluster_id],
        label=labels[cluster_id],
        s=30,
        alpha=0.5,
        edgecolors='none'
    )

# 4. HIGHLIGHT THE "IMPACT": Find and Label the Drifting Flight
# We look for the flight assigned to the 'Critical' cluster (Cluster 2)
drifting_flight_id = "FLIGHT-AX-104" # Use the specific ID you are monitoring

# If that specific flight exists in our data, let's highlight its path
if drifting_flight_id in pca_df['flight_id'].values:
    drift_data = pca_df[pca_df['flight_id'] == drifting_flight_id]

    # Draw a line showing the flight path "drifting" away from the healthy cluster
    ax.plot(drift_data['PC1'], drift_data['PC2'], drift_data['PC3'],
            color='black', linewidth=2, alpha=0.8, label=f"Path: {drifting_flight_id}")

```

```

_flight_id}")

    # Add a text annotation to the last point of that flight
    last_point = drift_data.iloc[-1]
    ax.text(last_point['PC1'], last_point['PC2'], last_point['PC3'],
            f" {drifting_flight_id} (DRIFT DETECTED)",
            color='black', fontweight='bold', fontsize=10)

# 5. Aesthetic Improvements & Labeling
ax.set_title("Edge Analytics Outcome: Predictive Pattern Recognition", font
size=16, pad=20)
ax.set_xlabel("PC1: System Stability Index", fontsize=12)
ax.set_ylabel("PC2: Engine Thermal Stress", fontsize=12)
ax.set_zlabel("PC3: Structural Vibration", fontsize=12)

# Custom Legend
ax.legend(loc='upper left', title="Fleet Status", frameon=True, shadow=True)

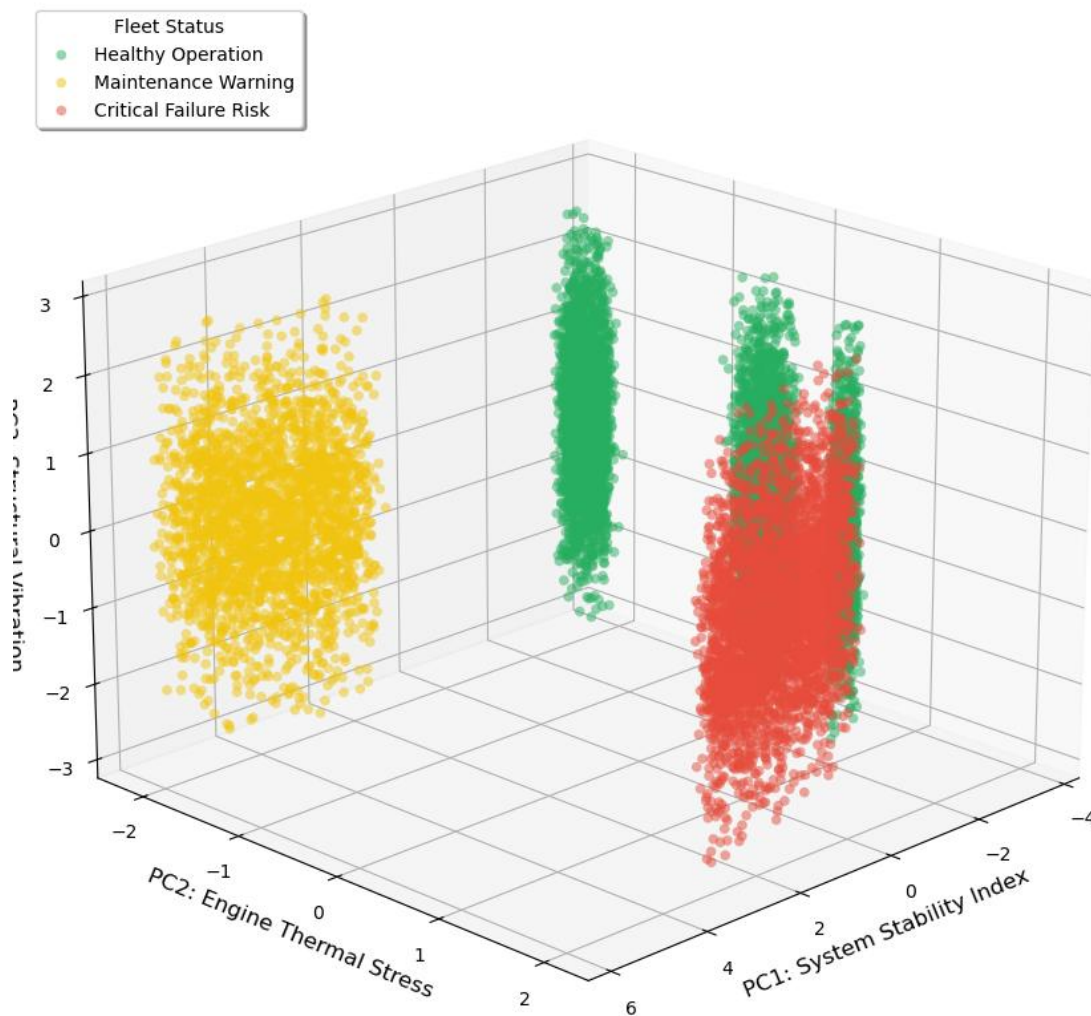
# Adjust viewpoint to best see the "drift"
ax.view_init(elev=20, azimuth=45)

plt.tight_layout()
plt.show()

# 6. Summary Output
print(f"--- Analytics Report ---")
critical_count = len(pca_df[pca_df['cluster'] == 2])
print(f"Alert: {critical_count} telemetry points identified in CRITICAL zone.")
print(f"Primary Target: {drifting_flight_id} shows significant deviation from nominal flight parameters.")

```

Edge Analytics Outcome: Predictive Pattern Recognition



--- Analytics Report ---

Alert: 3616 telemetry points identified in CRITICAL zone.

Primary Target: FLIGHT-AX-104 shows significant deviation from nominal flight parameters.

```
import pandas as pd
import json
```

```
# 1. Reload the raw data to get the text columns back
with open("balanced_flight_telemetry.json", "r") as f:
    raw_data = json.load(f)
```

```
df_original = pd.json_normalize(raw_data)
```

```
# 2. CRITICAL FIX: Filter df_original to only keep the rows that survived
your outlier removal
```

```
df_original = df_original.loc[df.index].copy()
```

```
# 3. Now copy the cluster labels (both dataframes are exactly 12,125 rows)
```



```

now!)
df_original['cluster_label'] = df['cluster_label'].values

# 4. Cross-tabulate Flights vs Clusters using the correct df_original variable
id_col = 'flight_info.callsign'
cluster_analysis = pd.crosstab(df_original[id_col], df_original['cluster_label'])

print("--- Analytics Outcome Report ---")
print("Flight Distribution across Clusters:")
print(cluster_analysis)
print("-" * 30)

# 5. Identify the "High Risk" Cluster
vibration_col = 'propulsion_sys.engine_1_vibration'

if vibration_col in df_original.columns:
    # Find cluster with max average vibration
    avg_vibrations = df_original.groupby('cluster_label')[vibration_col].mean()
    maintenance_cluster = avg_vibrations.idxmax()

    print(f"Average Vibration per Cluster:\n{avg_vibrations}\n")

    # 6. Identify specific flights with high presence in the Risk Cluster
    threshold = 50
    flagged_flights = cluster_analysis[cluster_analysis[maintenance_cluster] > threshold].index.tolist()

    print(f"[ALERT] Cluster {maintenance_cluster} identified as 'HIGH RISK' (Elevated Vibration).")

    if flagged_flights:
        print(f"[ACTION REQUIRED] Immediate inspection for: {flagged_flights}")
        print(f"Impact: Preventive maintenance on these units avoids potential engine failure.")
    else:
        print(f"[STATUS] No specific aircraft reached the critical threshold today.")
    else:
        print(f"Error: {vibration_col} not found in DataFrame. Check your column names.")

--- Analytics Outcome Report ---
Flight Distribution across Clusters:
cluster_label      0      1      2
flight_info.callsign
VIT-AI-400          1294  484  722
VIT-AI-500          1296  487  717
VIT-AI-600          1299  484  717
VIT-AI-700          1284  483  733
VIT-AI-800           912  486  727

```

Average Vibration per Cluster:

cluster_label

0 0.474790

1 0.979453

2 0.522632

Name: propulsion_sys.engine_1_vibration, dtype: float64

[ALERT] Cluster 1 identified as 'HIGH RISK' (Elevated Vibration).

[ACTION REQUIRED] Immediate inspection for: ['VIT-AI-400', 'VIT-AI-500', 'VIT-AI-600', 'VIT-AI-700', 'VIT-AI-800']

Impact: Preventive maintenance on these units avoids potential engine failure.